MINI PROJECT

SMART WASTE CLASSIFICATION

YUVASRI V 241501256

Aim

To develop a deep learning-based image classification system to identify the type of waste (**cardboard, glass, metal, paper, plastic or trash**) using a Convolutional Neural Network (CNN).

Project Idea

The project focuses on classifying waste from an input image (AI23531 – Deep Learning Mini Project, Google Colab).

The system takes an image of a waste item and classifies it into six categories:

- **Cardboard**
- **Glass**
- **Metal**
- **Paper**
- **Plastic**
- **Trash**

A deep learning model is used:

1. **CNN** – Model developed from scratch and trained on the TrashNet dataset.

The performance of the model is evaluated using **test accuracy**, and the trained CNN is used to predict the waste type of a sample or newly uploaded image.

Dataset

The public Kaggle **TrashNet** dataset (pre-resized version) is used. It is downloaded and extracted automatically in Colab.

Classes:

- Cardboard – 403 images

- Glass – 501 images
- Metal – 410 images
- Paper – 594 images
- Plastic – 482 images
- Trash – 137 images

The dataset is automatically divided class-wise into an 80/10/10 split:

- Training data – 2019 images, used for model training
- Validation data – 252 images, used to monitor training performance
- Testing data – 256 images, used for final evaluation

Images are resized to 128 × 128 pixels before being given to the model (batch size 32).

Model Training

Model – CNN

The CNN model contains:

- Image resizing and normalization (Rescaling 1/255)
- Convolution layers (3 × Conv2D – 32, 64 and 128 filters, ReLU)
- Max pooling layers
- Flatten layer
- Fully connected layer (Dense 128, ReLU)
- Dropout layer (0.4)
- Softmax output layer (6 classes)

The CNN has a total of 4,288,454 parameters (16.36 MB).

The CNN is trained using the Adam optimizer and Sparse Categorical Cross-Entropy loss.

Training uses:

- Up to 20 epochs
- EarlyStopping (monitor = val_loss, patience = 4, restore best weights)
- ModelCheckpoint (saves best model as best_waste_cnn.keras)

Model Training Code

The complete Colab code downloads the TrashNet dataset, splits it into training, validation and test sets, trains the CNN, plots the accuracy and loss graphs, generates a classification report and confusion matrix, saves the model, and predicts the waste type of a sample or uploaded image.

```
# STEP 1: Import libraries
import os, zipfile, shutil, random
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
print("TensorFlow:", tf.__version__)
print("GPU available:", bool(tf.config.list_physical_devices("GPU")))
```

```
# STEP 2: Automatically download and extract the TrashNet dataset
import urllib.request
import zipfile
from pathlib import Path
import shutil
# Direct URL to download the pre-resized TrashNet dataset
dataset_url = "https://github.com/garythung/trashnet/raw/master/data/dataset-resized.zip"
ZIP_PATH = Path("/content/trashnet.zip")
EXTRACT_DIR = Path("/content/waste_dataset")
print("Downloading TrashNet dataset (approx. 40MB)... This may take a moment.")
urllib.request.urlretrieve(dataset_url, ZIP_PATH)
print("Download complete.")
if EXTRACT_DIR.exists():
    shutil.rmtree(EXTRACT_DIR)
EXTRACT_DIR.mkdir(parents=True, exist_ok=True)
print("Extracting dataset...")
with zipfile.ZipFile(ZIP_PATH, "r") as z:
    z.extractall(EXTRACT_DIR)
```

```
# Move files up one level if they are wrapped in an extra nested folder
nested_folder = EXTRACT_DIR / "dataset-resized"
if nested_folder.exists():
    for item in nested_folder.iterdir():
        shutil.move(str(item), str(EXTRACT_DIR / item.name))
    shutil.rmtree(nested_folder)
print("Dataset successfully extracted to:", EXTRACT_DIR)

# STEP 3: Find the six class folders and prepare an 80/10/10 split
from pathlib import Path
import shutil
from sklearn.model_selection import train_test_split
if 'EXTRACT_DIR' not in locals() and 'EXTRACT_DIR' not in globals():
    EXTRACT_DIR = Path("/content/waste_dataset")
if 'SEED' not in locals() and 'SEED' not in globals():
    SEED = 42
CLASS_NAMES = ["cardboard", "glass", "metal", "paper", "plastic", "trash"]
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
class_dirs = {}
for cls in CLASS_NAMES:
    # Find matching directory
    matches = [p for p in EXTRACT_DIR.rglob(cls) if p.is_dir()]
    matches = [p for p in matches if any(f.suffix.lower() in IMAGE_EXTS for f in p.rglob("*")
    if f.is_file())]
    if not matches:
        # Check if the class images are directly under EXTRACT_DIR (flat extraction)
        direct_dir = EXTRACT_DIR / cls
        if direct_dir.is_dir() and any(f.suffix.lower() in IMAGE_EXTS for f in
        direct_dir.iterdir()):
            matches = [direct_dir]
    if not matches:
        raise FileNotFoundError(
            f"Could not find a folder named '{cls}' containing images under {EXTRACT_DIR}."
        )
    class_dirs[cls] = matches[0]
all_items = []
for cls, folder in class_dirs.items():
    images = [p for p in folder.rglob("*") if p.is_file() and p.suffix.lower() in IMAGE_EXTS]
    print(f"{cls}: {len(images)} images")
    all_items.extend((str(p), cls) for p in images)
```

```
if len(all_items) == 0:
    raise ValueError("No images found. Please check the dataset.")
# Split each class separately so all classes are represented in each set.
base = Path("/content/waste_split")
if base.exists():
    shutil.rmtree(base)
for split in ["train", "validation", "test"]:
    for cls in CLASS_NAMES:
        (base / split / cls).mkdir(parents=True, exist_ok=True)
for cls in CLASS_NAMES:
    paths = [p for p, label in all_items if label == cls]
    train_paths, temp_paths = train_test_split(paths, test_size=0.20, random_state=SEED)
    val_paths, test_paths = train_test_split(temp_paths, test_size=0.50, random_state=SEED)
    for split, items in [("train", train_paths), ("validation", val_paths), ("test", test_paths)]:
        for i, src in enumerate(items):
            dst = base / split / cls / f"{i:05d}_{Path(src).name}"
            shutil.copy2(src, dst)
print("\nSplit complete.")
for split in ["train", "validation", "test"]:
    print(split, "images:", sum(1 for p in (base / split).rglob("*") if p.is_file()))
```

```
# STEP 4: Load image datasets
from pathlib import Path
import tensorflow as tf
# Set correct base path to the successfully split dataset
base = Path("/content/waste_split")
SEED = 42
IMG_SIZE = (128, 128)
BATCH_SIZE = 32
if not (base / "train").exists():
    raise FileNotFoundError("Split dataset not found. Please run STEP 3 first.")
train_ds = tf.keras.utils.image_dataset_from_directory(
    base / "train",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int",
    seed=SEED
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    base / "validation",
```

```
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int",
    shuffle=False
)
test_ds = tf.keras.utils.image_dataset_from_directory(
    base / "test",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int",
    shuffle=False
)
class_names = train_ds.class_names
print("Classes successfully loaded:", class_names)
# Enable performance prefetches
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
test_ds = test_ds.prefetch(AUTOTUNE)

# STEP 5: Build the CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(128, 128, 3)),
    tf.keras.layers.Rescaling(1./255),
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(128, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(len(class_names), activation="softmax")
])
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
model.summary()

# STEP 6: Train the CNN
OUT_DIR = Path("/content/outputs")
OUT_DIR.mkdir(exist_ok=True)
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=4, restore_best_weights=True
    ),
    tf.keras.callbacks.ModelCheckpoint(
        str(OUT_DIR / "best_waste_cnn.keras"),
        monitor="val_loss", save_best_only=True
    )
]
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=callbacks
)

# STEP 7: Plot training and validation accuracy/loss
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training loss")
plt.plot(history.history["val_loss"], label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CNN Loss")
plt.legend()
plt.tight_layout()
plt.savefig(OUT_DIR / "training_graphs.png", dpi=150)
plt.show()
```

```
# STEP 8: Evaluate the trained model on the test set
test_loss, test_accuracy = model.evaluate(test_ds, verbose=1)
print(f"Test accuracy: {test_accuracy * 100:.2f}%")
print(f"Test loss: {test_loss:.4f}")
y_true = np.concatenate([labels.numpy() for _, labels in test_ds])
y_prob = model.predict(test_ds, verbose=1)
y_pred = np.argmax(y_prob, axis=1)
print("\nClassification report:")
print(classification_report(y_true, y_pred, target_names=class_names, zero_division=0))
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
fig, ax = plt.subplots(figsize=(8, 8))
disp.plot(ax=ax, xticks_rotation=45, colorbar=False)
plt.title("Confusion Matrix")
plt.tight_layout()
plt.savefig(OUT_DIR / "confusion_matrix.png", dpi=150)
plt.show()

# STEP 9: Save the final model and class labels
MODEL_PATH = OUT_DIR / "waste_cnn.keras"
model.save(MODEL_PATH)
with open(OUT_DIR / "class_names.txt", "w") as f:
    f.write("\n".join(class_names))
print("Saved model:", MODEL_PATH)
print("Saved files:", [p.name for p in OUT_DIR.iterdir()])

# STEP 10: Run prediction on a sample test image or upload your own
from PIL import Image
from google.colab import files
from pathlib import Path
import random
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
# Try to find a sample from our test split to predict automatically
test_images = list(Path("/content/waste_split/test").rglob("*"))
test_images = [p for p in test_images if p.is_file() and p.suffix.lower() in IMAGE_EXTS]
if test_images:
    image_path = random.choice(test_images)
```

```
    print(f"Selected sample test image: {image_path}")
    img = tf.keras.utils.load_img(image_path, target_size=IMG_SIZE)
else:
    print("Please upload a waste image (.jpg/.png) to test prediction:")
    new_upload = files.upload()
    image_files = [name for name in new_upload if Path(name).suffix.lower() in
    IMAGE_EXTS]
    if not image_files:
        raise ValueError("Please upload an image file such as JPG or PNG.")
    image_path = image_files[0]
    img = tf.keras.utils.load_img(image_path, target_size=IMG_SIZE)
arr = tf.keras.utils.img_to_array(img)
probs = model.predict(np.expand_dims(arr, axis=0), verbose=0)[0]
best_idx = int(np.argmax(probs))
plt.imshow(img)
plt.axis("off")
plt.title(f"Prediction: {class_names[best_idx]} ({probs[best_idx]*100:.1f}%)")
plt.show()
print("Predicted waste type:", class_names[best_idx])
print(f"Confidence: {probs[best_idx]*100:.2f}%")
print("\nAll class probabilities:")
for name, score in sorted(zip(class_names, probs), key=lambda x: x[1], reverse=True):
    print(f"{name}: {score*100:.2f}%")

# STEP 11 (optional): Download the trained model and output graphs
import shutil
from google.colab import files
from pathlib import Path
OUT_DIR = Path("/content/outputs")
shutil.make_archive("/content/smart_waste_outputs", "zip", OUT_DIR)
print("Archive created. Initiating file download...")
files.download("/content/smart_waste_outputs.zip")
```

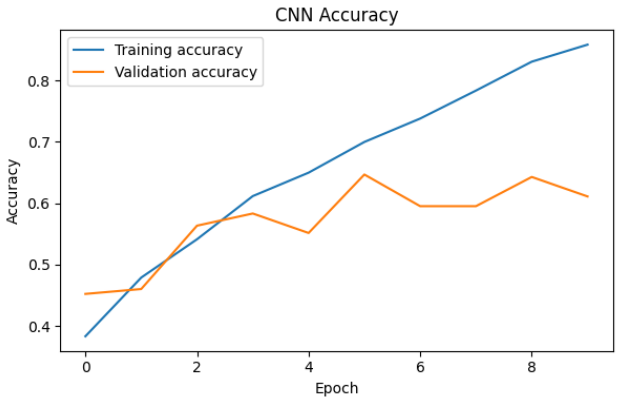
OUTPUT:

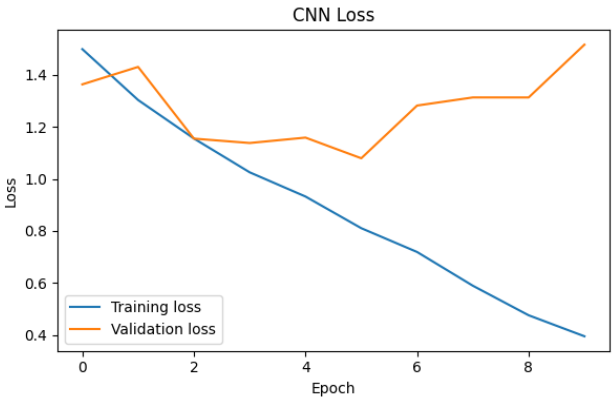
Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_1 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_2 (Conv2D)	(None, 32, 32, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0
flatten (Flatten)	(None, 32768)	0
dense (Dense)	(None, 128)	4,194,432
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 6)	774

Total params: 4,288,454 (16.36 MB)

```
Epoch 1/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 87s 1s/step - accuracy: 0.3834 - loss: 1.4989 - val_accuracy: 0.4524 - val_loss: 1.3634
Epoch 2/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 139s 1s/step - accuracy: 0.4789 - loss: 1.3032 - val_accuracy: 0.4603 - val_loss: 1.4305
Epoch 3/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 143s 1s/step - accuracy: 0.5414 - loss: 1.1552 - val_accuracy: 0.5635 - val_loss: 1.1548
Epoch 4/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 83s 1s/step - accuracy: 0.6117 - loss: 1.0252 - val_accuracy: 0.5833 - val_loss: 1.1379
Epoch 5/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 82s 1s/step - accuracy: 0.6498 - loss: 0.9322 - val_accuracy: 0.5516 - val_loss: 1.1587
Epoch 6/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 83s 1s/step - accuracy: 0.6999 - loss: 0.8103 - val_accuracy: 0.6468 - val_loss: 1.0792
Epoch 7/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 82s 1s/step - accuracy: 0.7380 - loss: 0.7189 - val_accuracy: 0.5952 - val_loss: 1.2819
Epoch 8/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.7836 - loss: 0.5891 - val_accuracy: 0.5952 - val_loss: 1.3131
Epoch 9/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 84s 1s/step - accuracy: 0.8306 - loss: 0.4755 - val_accuracy: 0.6429 - val_loss: 1.3130
Epoch 10/20
64/64 ━━━━━━━━━━━━━━━━━━━━ 82s 1s/step - accuracy: 0.8583 - loss: 0.3948 - val_accuracy: 0.6111 - val_loss: 1.5161
```



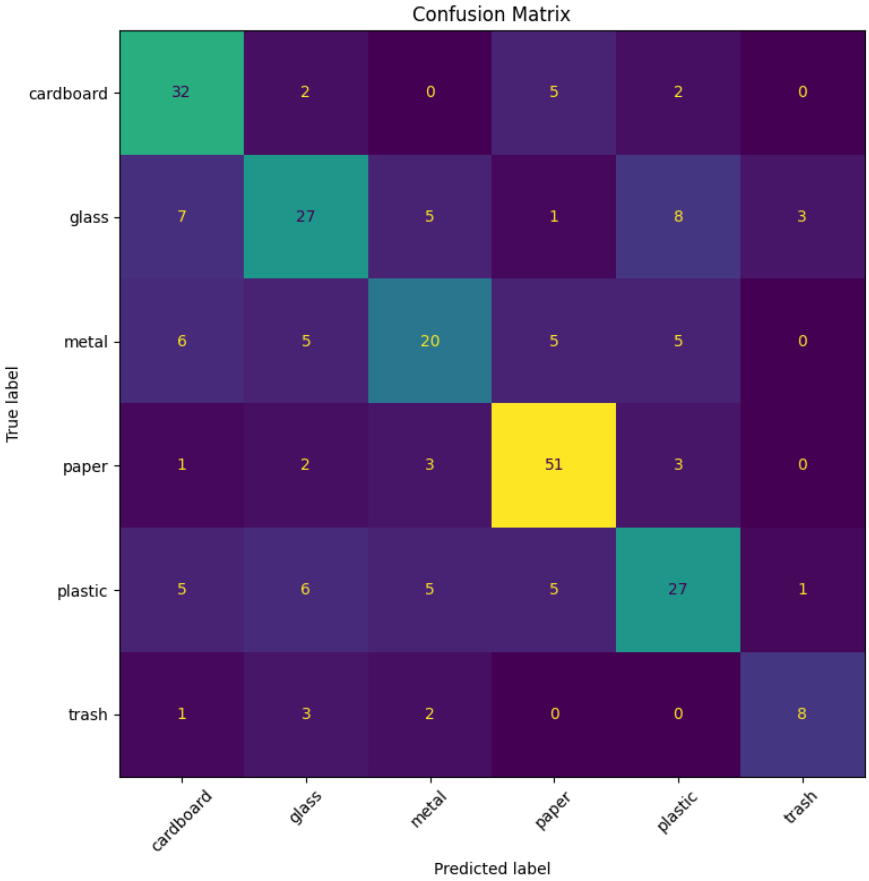



```
Test accuracy: 64.45%
Test loss: 1.0515
8/8 ━━━━━━━━━━━━━━━━━━━━ 3s 321ms/step

Classification report:
              precision    recall  f1-score   support

   cardboard       0.62      0.78      0.69        41
       glass       0.60      0.53      0.56        51
       metal       0.57      0.49      0.53        41
       paper       0.76      0.85      0.80        60
     plastic       0.60      0.55      0.57        49
       trash       0.67      0.57      0.62        14

    accuracy                           0.64       256
   macro avg       0.64      0.63      0.63       256
weighted avg       0.64      0.64      0.64       256
```


Confusion Matrix
cardboard
glass
metal
paper
plastic
trash
True label
Predicted label
32 2 0 5 2 0
7 27 5 1 8 3
6 5 20 5 5 0
1 2 3 51 3 0
5 6 5 5 27 1
1 3 2 0 0 8

```
Selected sample test image: /content/waste_split/test/metal/00026_metal191.jpg
```




```
Predicted waste type: paper
Confidence: 52.44%

All class probabilities:
paper: 52.44%
metal: 47.39%
plastic: 0.07%
glass: 0.07%
trash: 0.02%
cardboard: 0.01%
```

Result

The CNN model was successfully trained to classify waste images into six categories: cardboard, glass, metal, paper, plastic and trash. The performance of the model was evaluated using test accuracy (64.45%) and test loss (1.0515). The model was also evaluated using accuracy and loss graphs, classification report, and confusion matrix.